

RESIDUALQUANT: KV Cache Quantization for Looped Transformers with 2-Bit Residuals

Heejun Kim¹, Junyoung Lee², Sanglyul Cho³, Dongsu Han¹, Insu Han¹ and Sehoon Kim^{1,†}

¹KAIST, ²Yonsei University, ³Seoul National University

Abstract: Looped Transformers extend computational depth through weight sharing, but their per-loop KV caches consume the memory saved by shared weights, limiting batch capacity and inference throughput. We propose RESIDUALQUANT, a residual quantization method that exploits predictability across loops. RESIDUALQUANT uses the final loop within each loop group as a shared anchor, predicts the other loops’ KV states with least-squares scaling, and quantizes the remaining residuals. Shared rotations and loop-wise precision allocation enable INT4 anchors and INT2 residuals without retaining a separate BF16 region for past KV states. We evaluate RESIDUALQUANT on Ouro-1.4B and Huginn-3.5B across mathematical reasoning and code generation benchmarks. The results show that residual quantization achieves higher accuracy at low effective bitwidths than direct quantization or rotation alone. On Ouro-1.4B, RESIDUALQUANT reduces logical KV storage by 80.7% relative to BF16 while achieving 77.6% accuracy on MATH500. In a separate 8k-context experiment on an RTX 5090, RESIDUALQUANT supports $4\times$ the largest feasible tested power-of-two batch and delivers $3.27\times$ the peak decode throughput of BF16. We further demonstrate the benefits of least-squares residuals for W4A4 activation quantization.

🌐 Project page: <https://seal-kaist.github.io/projects/ResidualQuant>

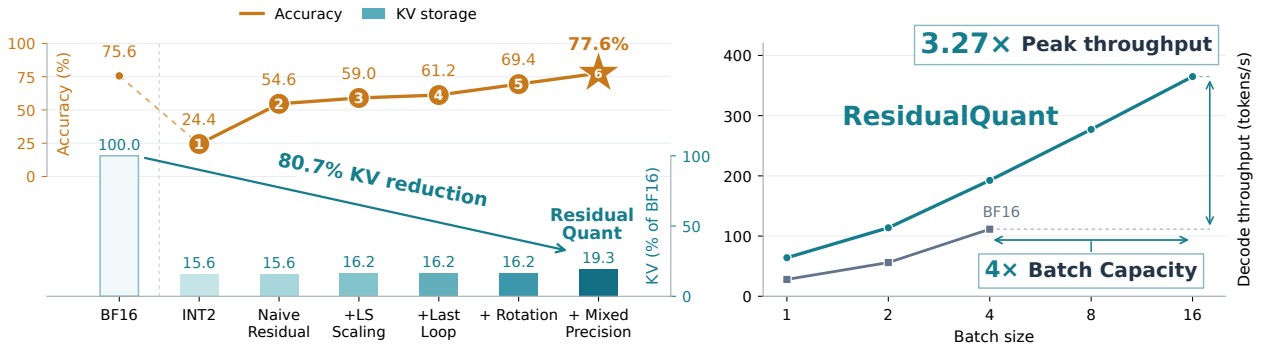


Figure 1: RESIDUALQUANT progressively recovers BF16-level accuracy while reducing KV storage and improving decode throughput. Results on Ouro-1.4B with group size $g = 32$ in every quantized loop. Left: Starting from direct INT2 ①, we successively introduce Previous-loop residual quantization ②, Least Square scaling ③, Last-loop references ④, shared rotation ⑤, and mixed precision ⑥ (RESIDUALQUANT). Accuracy increases from 24.4% to 77.6%, exceeding the BF16 baseline of 75.6%, while logical KV storage is reduced by 80.7%. Right: At 8k context on an RTX 5090, RESIDUALQUANT achieves $3.27\times$ the peak decode throughput and $4\times$ the largest feasible tested power-of-two batch of BF16.

1. Introduction

Recursive or looped Transformers refine latent representations by repeatedly applying shared modules, increasing computational depth without increasing the parameter count (Dehghani et al., 2019; Saunshi et al., 2025; Suleymanzade et al., 2026; Wang et al., 2026b). Models such as Huginn (Geiping et al., 2025) and Ouro (Zhu et al., 2025) exploit this structure through adjustable recurrence or adaptive depth allocation, requiring less weight storage than models with distinct parameters at the same computational depth. However, each loop produces distinct keys and values, making KV cache memory grow with recurrent depth, in addition to context length and batch size, despite weight sharing. This limits the extent to which weight savings translate into larger batches and higher throughput, making KV-cache memory reduction an important challenge for realizing the full efficiency benefits of looped Transformers.

† Corresponding author: Sehoon Kim.

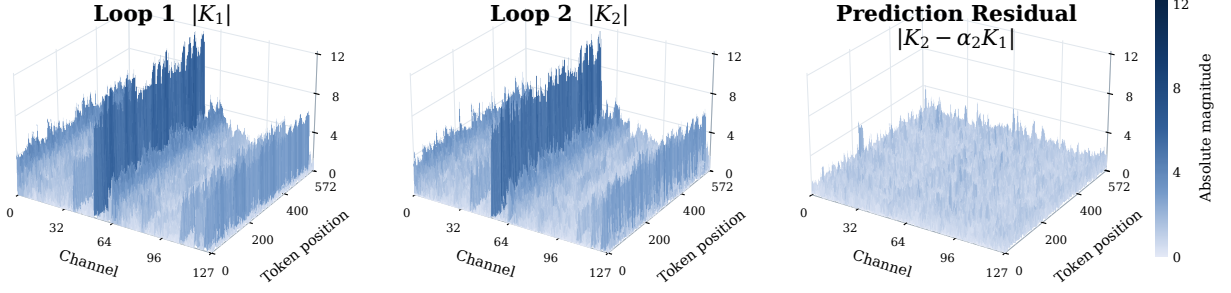


Figure 2: Post-RoPE key magnitudes in Ouro-1.4B. Left: Loop 1, $|K_1|$. Middle: Loop 2, $|K_2|$. Right: the residual after Least Square scaling of Loop 1’s keys, $|K_2 - \alpha_2 K_1|$. All panels use the same height and color scales. Visualization details and last-loop anchor comparisons are provided in Appendix C.

To reduce this storage overhead, prior work has explored sharing or reusing KV states across recurrences. MoR (Bae et al., 2025b) reuses KV states from the first recursion, PLT (Wu et al., 2025) combines first-loop global KV with local attention in subsequent loops, and MELT (Vendrell et al., 2026) updates shared KV states through learned gates. While these approaches reduce storage, sharing KV states across loops can discard loop-specific information and degrade accuracy when representations differ across loops.

KV quantization offers an alternative approach that preserves distinct KV states for each loop while reducing their storage precision. Because KV memory grows with recurrent depth, aggressive low-precision quantization is particularly desirable for looped Transformers. However, existing KV quantization methods (Hooper et al., 2024; Liu et al., 2024), including state-of-the-art rotation-based approaches (Ashkboos et al., 2024; Yun et al., 2026; Zhou et al., 2026), often suffer substantial accuracy degradation in sub-4-bit regimes. Looped Transformers, however, present a unique opportunity: KV states from different loops are closely related because they are generated by repeatedly applying the same shared modules. This creates strong similarity between KV states across loops as shown in Figure 2, which can be exploited to improve low-bit quantization

Motivated by this *inter-loop similarity*, we propose RESIDUALQUANT, a loop-aware KV cache quantization method that represents each loop’s KV states as *residuals* from those of a reference *anchor* loop. The resulting residuals (e.g., Figure 2, right) are more robust to low-bit quantization, enabling aggressive KV cache compression using INT2 precision while preserving loop-specific information. Our main contributions can be summarized as follows:

- We introduce RESIDUALQUANT, which reconstructs loop-specific KV states using the final-loop KV states as an anchor and stores their differences as low-precision residuals (Section 3.2). We further apply least-squares residual scaling, learned rotations (Section 3.3), and loop-wise INT4/INT2 mixed precision (Section 3.4), which together enable an effective balance between low-bit quantization accuracy and runtime efficiency.
- We show that RESIDUALQUANT substantially improves the accuracy–memory tradeoff over state-of-the-art rotation-based KV quantization across Ouro-1.4B and Huginn-3.5B on reasoning and code generation benchmarks. In particular, our INT2/4 configuration reduces KV memory by 80.7% relative to BF16 while matching BF16 accuracy, whereas baseline methods degrade sharply at the same KV cache budget (Section 5.1).
- We demonstrate that these KV memory savings translate directly into higher inference throughput. At 16k context on an RTX 5090, RESIDUALQUANT improves fixed-batch decode throughput by $2.73\times$ through reduced KV cache memory traffic. Its smaller KV memory footprint further increases the largest feasible tested batch size by $2\times$, yielding a $4.15\times$ improvement in peak decode throughput over BF16 (Section 5.3).
- We further show that the same residual reconstruction principle extends to activation quantization, consistently improving accuracy over existing quantization methods, including state-of-the-art weight–activation quantization method like LoopQ (Fang et al., 2026) and FlatQuant (Sun et al., 2025) (Section 5.4).

2. Preliminary

2.1. Group-wise Affine Quantization

Low-bit quantization requires adapting the quantization range to the value distribution. Group-wise scales and offsets control the spacing and starting point of quantization levels. For each token and KV head, we partition the channel dimension into groups $\mathcal{G}_j$ of size g , with each group sharing a scale and an offset. Affine quantization with stored scale $\bar{s}_j > 0$ and offset $\bar{o}_j$ is given by (Bhalgat et al., 2020; Liu et al., 2024; Yun et al., 2026)

$$c_i = \text{clip}_{[0, 2^b - 1]} \left(\text{round} \frac{x_i - \bar{o}_j}{\bar{s}_j} \right), \quad \hat{x}_i = \bar{o}_j + \bar{s}_j c_i, \quad i \in \mathcal{G}_j. \quad (2.1)$$

Each code c_i is stored in b bits, while elements in the same group share the scale and offset. We use Q to denote quantization followed by reconstruction using these codes and metadata.

At extreme quantization such as INT2, zero-centered symmetric quantization, where the offset is fixed to zero and therefore does not need to be stored, can use only three distinct reconstruction levels, $\{-\bar{s}_j, 0, \bar{s}_j\}$, despite having four available codes. Affine quantization allows the reconstruction levels to shift through $\bar{o}_j$, using all four levels, $\{\bar{o}_j, \bar{o}_j + \bar{s}_j, \bar{o}_j + 2\bar{s}_j, \bar{o}_j + 3\bar{s}_j\}$, without requiring symmetry around zero. Therefore, throughout this work, we adopt group-wise affine quantization to make full use of the limited INT2 code space while adapting the range to each group’s distribution.

2.2. Rotation-based Quantization

In Equation 2.1, outliers can increase the group scale $\bar{s}_j$, widening the spacing between quantization levels and reducing resolution for smaller values. Orthogonal rotations are introduced to redistribute these large magnitudes across channels to mitigate this effect (Chee et al., 2023; Ashkboos et al., 2024). Given an orthogonal matrix U , quantization and reconstruction can be expressed as

$$Q^U(X) = U^\top Q(UX), \quad \|UX\|_2 = \|X\|_2. \quad (2.2)$$

Existing approaches range from randomized Hadamard rotations (Ashkboos et al., 2024) to learned rotations (Liu et al., 2025). For INT2 KV cache quantization, we choose OptR (Yun et al., 2026), which optimizes head-wise orthogonal rotations to minimize attention output error. Specifically, we use its Hadamard-initialized variant, OptR-H, to obtain U for inter-loop residual quantization (Section 3.3).

3. Methodology

In this section, we present RESIDUALQUANT, our loop-wise residual quantization method for looped Transformers. We first motivate inter-loop residual quantization (Section 3.1). Starting from direct INT2 KV cache quantization, we then progressively improve performance by introducing residual quantization with least square (LS) scaling (Section 3.2), rotation of the residuals (Section 3.3), and loop-wise mixed precision (Section 3.4), as illustrated in Figure 1. Together, these components form RESIDUALQUANT, which recovers BF16-level performance across the evaluated models and benchmarks (Table 8).

3.1. Motivation

Looped Transformers repeatedly process the same token through shared blocks and apply the same key and value projections across loops. At a fixed layer, token, and head, write $X^{(r)} \in \{K^{(r)}, V^{(r)}\}$ as $X^{(r)} = W_X z^{(r)}$, where $z^{(r)}$ is the preprocessed input and W_X is the projection shared across loops. It follows that

$$\|X^{(r)} - X^{(r-1)}\|_2 \leq \|W_X\|_2 \|z^{(r)} - z^{(r-1)}\|_2. \quad (3.1)$$

Thus, when recurrent inputs change little between loops, the corresponding KV differences are bounded by this change scaled by $\|W_X\|_2$. The fixed-point perspective of deep equilibrium models (Bai et al., 2019) provides one explanation for small changes across iterations, while Huginn-3.5B exhibits input-dependent trajectories approaching fixed points or forming orbits (Geiping et al., 2025).

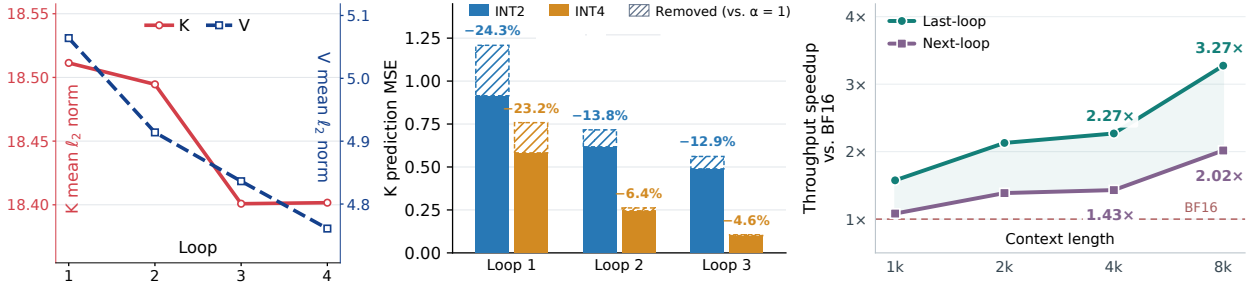


Figure 3: Loop-wise KV statistics and the effects of prediction and reference choices on Ouro-1.4B. Left: Mean K and V norms across loops. Middle: Key prediction MSE using Last-loop references under uniform INT2 and INT4, with group size $g = 32$ and no rotation. Solid bars show prediction MSE with LS scaling, while hatched portions indicate the additional error without LS scaling ($\alpha = 1$). Right: Decode throughput with INT2/4 and $g = 32$ on an RTX 5090, comparing the Last-loop and Next-loop reference policies with OptR-H rotations. For each context length, each method runs at its maximum feasible batch size, with speedups normalized to BF16 FlashAttention at the same batch size. Analysis details are provided in Appendix B.

In practice, Figure 2 empirically shows that key entries exhibit similar patterns across loops, allowing one loop’s keys to be represented using another with substantially smaller *inter-loop residuals*. As shown in the figure, these residuals contain far fewer outliers than the original key values themselves and are therefore much easier to quantize. Together, the shared-projection structure and this observation motivate quantizing residuals instead of each loop’s KV states independently. We therefore use a reference KV state to capture shared information and store loop-specific differences as low-precision residuals.

3.2. Residual Quantization for KV Cache

With direct INT2 affine quantization at group size $g = 32$, Ouro-1.4B achieves only 24.4% MATH500 accuracy (Figure 1). Motivated by Section 3.1, we reconstruct each loop’s KV using a reference KV state and quantize only the residual. For loop r , let $X_r \in \mathbb{R}^D$ denote a key or value vector at a fixed layer, token, and head. A reference R_r from the stored cache and a scalar coefficient α_r give

$$\hat{X}_r = \alpha_r R_r + Q(X_r - \alpha_r R_r), \quad (3.2)$$

Thus, instead of storing X_r directly, we store only its quantized residual, i.e., $Q(X_r - \alpha_r R_r)$, and reconstruct $\hat{X}_r \simeq X_r$ on demand using the reference R_r .

The most intuitive choice is to use the immediately preceding loop as the reference and directly quantize the inter-loop difference $X_r - \hat{X}_{r-1}$. This is when $R_r = \hat{X}_{r-1}$ and $\alpha_r = 1$. The first loop is quantized independently, i.e., $\hat{X}_0 = Q(X_0)$, and serves as an anchor for reconstructing subsequent loops. This simple approach improves MATH500 accuracy from 24.4% to 54.6% under uniform INT2 quantization (① $\rightarrow$ ② in Figure 1). However, we find this choice *suboptimal* and instead jointly select the scaling coefficient α_r and reference KV R_r to further improve model quality while retaining efficient reconstruction.

Scaling coefficient α_r . Figure 3 (left) shows that KV magnitudes vary across Ouro-1.4B’s loops, motivating a scaling coefficient rather than directly subtracting the reference with $\alpha_r = 1$. We use the LS coefficient $\alpha_r = \langle X_r, R_r \rangle / \|R_r\|_2^2$ to minimize $\|X_r - \alpha_r R_r\|_2^2$. Figure 3 (middle) shows that, using the same reconstructed reference, LS reduces key prediction MSE measured before residual quantization relative to $\alpha_r = 1$. With Previous-loop references and uniform INT2, LS scaling improves MATH500 accuracy from 54.6% to 59.0% (② $\rightarrow$ ③ in Figure 1).

Reference KV R_r . While the *Previous-loop* policy, which reconstructs each loop from its predecessor KV state, is intuitive, we propose to reverse the reconstruction direction that uses the successor as the reference, i.e.,

Table 1: Reference direction on Ouro-1.4B. Uniform INT2 with LS scaling. NMSE compares original and reconstructed keys on all 500 MATH500 problems (Appendix B).

Reference	Key NMSE $\times 10^3 \downarrow$			
	Loop 1	Loop 2	Loop 3	Loop 4
Previous-loop	179.38	34.67	10.39	4.80
Next-loop	21.54	8.69	18.24	178.21

$R_r = \hat{X}_{r+1}$. This *Next-loop* policy is motivated by the importance of preserving early-loop KV states, which can be more precisely reconstructed from the final-loop anchor through multiple successive residual corrections. Table 1 compares the two directions under uniform INT2 with LS scaling. *Next-loop* reduces reconstruction error in early loops, which can translate into higher accuracy, as shown in the ablation comparing different reference choices (Table 2 in Section 5.2).

The *Next-loop* policy, however, requires loading multiple residuals to reconstruct each loop’s KV. For example, in a 4-loop model such as Ouro, reconstructing loop 1 requires loading the final-loop anchor and the residuals of loops 3, 2, and 1, resulting in four KV cache reads compared to just one with direct quantization. This increases overall memory traffic and limit achievable decoding throughput. To address this overhead, we propose the *Last-loop* policy, which directly references the final-loop anchor, $R_r = \hat{X}_L$ for $r = 0, \dots, L - 1$. This allows each non-anchor loop to load only the anchor and its own residual. Figure 3 (right) shows that Last-loop achieves higher decode throughput than Next-loop by up to $1.62\times$. As shown later in Table 2 of Section 5.2, Last-loop also retains accuracy comparable to Next-loop. We therefore adopt Last-loop in RESIDUALQUANT to improve decoding efficiency without sacrificing accuracy as shown in Figure 1 (③ $\rightarrow$ ④). Detailed reconstruction and access-count comparisons are provided in Appendix D.

With the *Last-loop* policy, the current token’s anchor is unavailable until the final loop completes. We therefore retain the current token’s KV states in BF16 during decoding and defer quantization and cache storage until the final loop completes, as described in Section 4.2.

3.3. Rotation for KV Cache Residual

Residual quantization with LS scaling and the Last-loop policy effectively mitigates KV outliers, but some outliers can still remain in the residuals (Figure 2, right). We further reduce their impact by applying an orthogonal rotation U to the residual $X_r - \alpha_r R_r$, rather than to the KV state X_r itself, thereby redistributing large residual values across channels. We quantize the rotated residual and apply the inverse rotation during reconstruction. Incorporating rotation into Equation 3.2 gives

$$\hat{X}_r = \alpha_r R_r + U^\top Q(U(X_r - \alpha_r R_r)), \quad U^\top U = I. \quad (3.3)$$

We use OptR-H (Yun et al., 2026) to learn separate rotations for keys and values at each layer and head, sharing them across all loops. These fixed matrices are reused across tokens, so their storage does not grow with sequence length. Applying rotation to the INT2 residual quantization obtained in Section 3.2 further improves Ouro-1.4B’s MATH500 accuracy from 61.2% to 69.4%, with an 8.2% gain (④ $\rightarrow$ ⑤ in Figure 1).

3.4. Loop-wise Mixed Precision

So far, our proposed techniques have improved INT2 MATH500 accuracy to 69.4%, substantially outperforming direct quantization (24.4%) and rotation-based quantization using OptR alone (33.2%, Table 8). To further close the remaining accuracy gap from BF16, we exploit the fact that, under the *Last-loop* policy, the final-loop anchor stores a full KV state that serves as the shared reference for reconstructing all other loops. Since the quality of this shared reference directly affects the reconstruction of all residual loops, we allocate INT4 to the anchor while retaining INT2 for the residuals. For Ouro-1.4B with four loops, this yields a precision configuration of $[2, 2, 2, 4]$, increasing the average bitwidth from 2 to 2.5, excluding metadata. Including quantization metadata, this configuration achieves a $5.2\times$ reduction in KV memory footprint relative to BF16.

This loop-wise mixed-precision allocation improves MATH500 accuracy from 69.4% with uniform INT2 to 77.6%, matching the BF16 accuracy with only a 19% increase in KV-cache storage (⑤ $\rightarrow$ ⑥ in Figure 1). Combining LS scaling, the Last-loop policy, rotation-based quantization, and anchor-focused mixed precision, we obtain our proposed method, RESIDUALQUANT.

4. Implementation

Accelerating RESIDUALQUANT requires ensuring that reconstruction costs do not offset the memory accesses saved by compression. Referencing the final loop also requires handling the current token before its anchor

becomes available. We address these requirements with attention kernels that consume compressed KV directly and a vLLM (Kwon et al., 2023) integration that defers cache storage until the final loop completes.

4.1. Kernel Implementation

To avoid writing reconstructed KV to GPU memory and reading it back, our custom CUDA attention kernel reconstructs anchors and residuals tile by tile and immediately uses them for QK dot products and weighted V accumulation. With the Last-loop policy, each residual loop requires only the anchor and its own residual, avoiding sequential reconstruction of intermediate loops.

We further reduce repeated reconstruction work by reusing group-wise scales and offsets. For each token and quantization group in a KV tile, we precompute the possible dequantized V residual values in a shared-memory lookup table. For INT2 residuals, each table contains the four values corresponding to integer codes 0–3. Channels sharing the same quantization parameters use their codes to index the table, avoiding repeated scale-and-offset computations. Asynchronous loads overlap the next tile’s memory accesses with computation on the current tile. Because anchors and residuals share a rotation, we reuse the rotated Q across both components and apply a single inverse rotation after combining their weighted V outputs. Softmax and attention accumulation use FP32.

4.2. vLLM Integration

We integrate a custom attention backend for compressed caches while retaining vLLM’s request scheduling and paged KV management. Triton kernels handle KV storage, with separate prefill and decode paths.

Prefill. Each loop computes causal attention using its BF16 prompt KV states. After the final loop completes, we quantize its KV states as the anchor and use the reconstructed anchor to compute residuals with LS scaling for the non-anchor loops. The anchor and residuals are stored as packed codes and metadata for subsequent decoding.

Decode. For past tokens, the custom CUDA kernel reads the compressed cache, reconstructs KV, and computes attention. Since the current token’s last-loop anchor is not yet available, each loop uses its current BF16 KV states, while those from the non-anchor loops are held in a temporary buffer. A Triton kernel combines the attention contributions from past and current tokens using their softmax normalization statistics. Once the final loop completes, we quantize the current token’s anchor, compute the residuals for the non-anchor loops with LS scaling, and store them in the cache. This decode path runs with CUDA Graphs.

5. Experiments

Building on the MATH500 improvements presented in Section 3, we evaluate whether RESIDUALQUANT maintains accuracy across multiple benchmarks and experimental settings and translates KV storage savings into higher throughput. The evaluation begins with accuracy and KV storage comparisons across different models (Section 5.1), followed by component-wise ablations of RESIDUALQUANT (Section 5.2). Throughput measurements assess the practical gains from KV compression (Section 5.3), and activation quantization experiments test the broader applicability of residual quantization (Section 5.4).

Models. We evaluate RESIDUALQUANT on Ouro-1.4B and Huginn-3.5B. Ouro-1.4B executes four loops, while Huginn-3.5B executes 32 loops. For Huginn-3.5B, we partition the 32 loops into eight groups of four and apply RESIDUALQUANT independently within each group, using the final loop of each group as its anchor.

Quantization setup. For KV quantization experiments, model weights remain in BF16, and we evaluate KV cache in BF16, uniform INT2, uniform INT4, and loop-wise mixed precision INT2/4. Unless otherwise specified, we use group size $g = 32$, and for INT2/4, allocate INT4 only to the anchor loop, following Section 3.4. The main results in Section 5.1 and the ablations in Section 5.2 also evaluate INT2 with $g = 16$. Since the Last-loop implementation of our method keeps the current token’s KV states in BF16 during inference, we apply the same treatment to all baseline methods for a fair comparison. As baselines, we consider direct uniform quantization

and the rotation-based OptR-H method (Yun et al., 2026). Rotation calibration and inference details are provided in Appendix H.

Benchmarks. For mathematical reasoning, we report accuracy on GSM8K (Cobbe et al., 2021) and MATH500 (Lightman et al., 2024). For code generation, we report pass@1 on HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021). Prompt formats, generation limits, and other evaluation details are provided in Appendix A.2.

5.1. Main Results

The main evaluation examines the accuracy–memory tradeoff of KV-cache quantization using the effective bitwidth b_{eff} , which measures the total KV cache storage cost per each KV element. While b_{eff} is determined primarily by the quantization precision, it also accounts for auxiliary metadata such as quantization scales and offsets, as well as the scaling coefficients introduced by LS scaling. Details on how b_{eff} is measured are provided in Appendix A.3.

We compare direct uniform quantization, OptR-H, and our full RESIDUALQUANT method on Ouro-1.4B and Huginn-3.5B. We additionally evaluate RESIDUALQUANT without rotation to isolate the contribution of residual quantization itself. For each method, we vary the KV cache precision to obtain multiple configurations along the accuracy–memory tradeoff: INT4, mixed-precision INT2/4, and INT2 with group sizes $g = 16$ and $g = 32$.

Figure 4 shows that RESIDUALQUANT consistently improves the accuracy–memory tradeoff across models and benchmarks. RESIDUALQUANT *without* rotation strictly outperforms the state-of-the-art rotation-based OptR-H baseline at most KV cache budgets across all models and benchmarks, demonstrating that exploiting inter-loop residual structure is itself an effective way to quantize looped transformer KV caches. Rotation provides a complementary benefit: combining residual quantization with OptR-H, i.e., the full RESIDUALQUANT method, further improves the tradeoff, with the gains becoming particularly pronounced in the low-precision regime where direct quantization and OptR-H degrade sharply.

Under the INT2/4 mixed-precision configuration, RESIDUALQUANT matches BF16 accuracy across all evaluated cases while reducing KV memory by 80.7%. At the most aggressive configuration, using uniform INT2 with group size $g = 32$, the advantage becomes more pronounced. On Ouro-1.4B, RESIDUALQUANT improves over OptR-H from 41.70% to 69.75% on GSM8K and from 33.20% to 69.40% on MATH500. On Huginn-3.5B, the corresponding improvements are from 24.72% to 37.30% and from 8.80% to 13.80%, respectively. The results are particularly strong on code generation under this configuration, where RESIDUALQUANT matches BF16 accuracy across both models. Full results are provided in Appendix F.

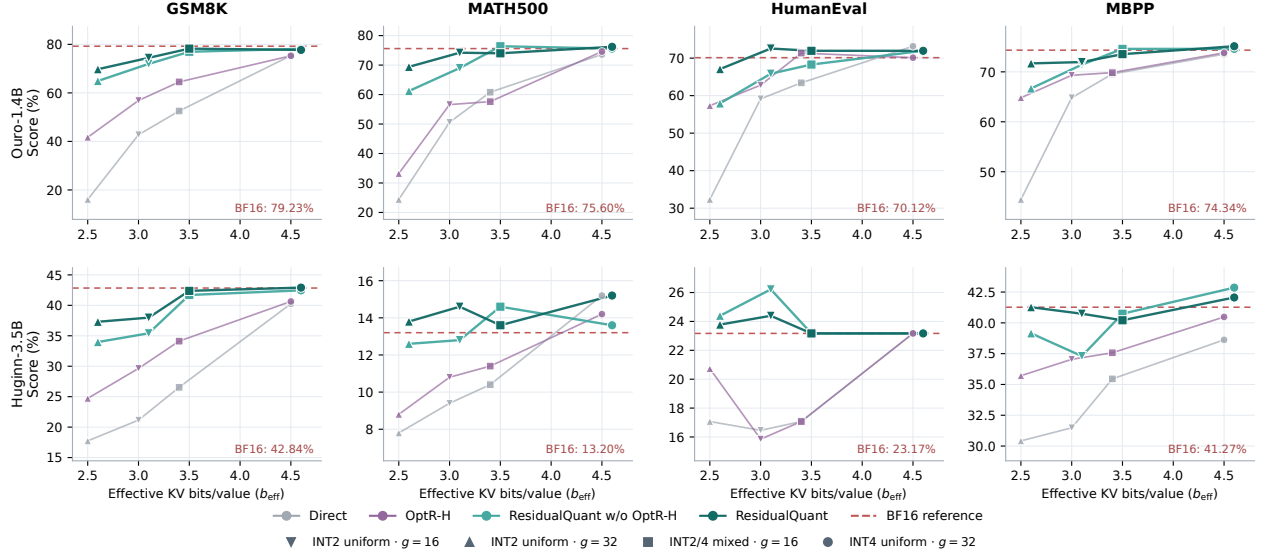


Figure 4: Accuracy versus effective KV bitwidth across models and benchmarks. Colors denote methods, and markers denote quantization schedules and group sizes. Effective bitwidth includes quantized values and metadata. Red dashed lines indicate BF16 accuracy. Full results are provided in Appendix F.

Table 2: LS scaling and reference selection ablation on Ouro-1.4B. MATH500 accuracy (%) without rotation. Direct quantization baselines are 50.6%, 73.6%, and 47.0% for INT2, INT4, and INT2/4, respectively. INT2/4 denotes the [2, 2, 2, 4] mixed-precision configuration. For each precision setting, we evaluate residual quantization with and without LS scaling across four reference-selection policies: Previous-loop, Next-loop, First-loop, and Last-loop. Colored parentheses show absolute changes from the corresponding direct baseline.

Reference	INT2		INT4		INT2/4	
	w/o LS scaling	w/ LS scaling	w/o LS scaling	w/ LS scaling	w/o LS scaling	w/ LS scaling
Previous-loop	63.6 (+13.0)	69.6 (+19.0)	73.4 (−0.2)	73.0 (−0.6)	65.4 (+18.4)	68.2 (+21.2)
Next-loop	68.2 (+17.6)	69.2 (+18.6)	75.4 (+1.8)	76.2 (+2.6)	72.2 (+25.2)	72.6 (+25.6)
First-loop	38.6 (−12.0)	62.6 (+12.0)	73.2 (−0.4)	74.6 (+1.0)	50.8 (+3.8)	66.0 (+19.0)
Last-loop	67.6 (+17.0)	69.0 (+18.4)	75.6 (+2.0)	75.6 (+2.0)	64.6 (+17.6)	72.8 (+25.8)

Table 3: Complementary effects of residual quantization and rotation. MATH500 accuracy (%) on Ouro-1.4B with uniform INT2 and INT4. Direct denotes standard uniform quantization, while Residual uses LS scaling with the Last-loop reference policy. OSCAR (Zhou et al., 2026) and OptR-H (Yun et al., 2026) are applied to direct or residual quantization. BF16 accuracy is 75.6%.

Precision	Direct	+ OSCAR	+ OptR-H	Residual	+ OSCAR	+ OptR-H
INT2	50.6	56.0	56.6	69.0	74.6	74.2
INT4	73.6	75.6	74.6	75.6	76.0	76.2

5.2. Ablation Studies

We further evaluate the contributions of the individual components introduced in Section 3 through ablations of RESIDUALQUANT, including LS scaling, reference choice, rotation, and loop-wise precision allocation. All accuracy comparisons use Ouro-1.4B on MATH500. A full table comparing multiple scaling methods, rotations, and reference choices is provided in Appendix G.

LS scaling. Table 2 compares residual quantization with and without LS scaling across four reference selection policies and three precision settings (INT2, INT4, and INT2/4). LS scaling improves accuracy in most configurations, demonstrating its effectiveness across different reference choices and precisions. In particular, under the INT2/4 Last-loop configuration, LS scaling improves MATH500 accuracy from 64.6% to 72.8%, an 8.2% gain. This improvement is also supported by Figure 3 (middle), where LS scaling effectively reduces key prediction MSE by up to 24.3% relative to the unscaled case.

Reference choice. Table 2 also demonstrates the impact of reference selection. Compared with Previous-loop, which is the most intuitive reconstruction policy, Last-loop improves accuracy in most cases, by up to 4.6 percentage points under INT2/4 with LS scaling. Even compared with Next-loop, Last-loop achieves nearly identical accuracy across all settings especially with LS scaling. Importantly, Last-loop is more runtime-efficient than both Previous-loop and Next-loop because each loop requires fewer KV-cache loads, yielding up to a $1.62\times$ throughput improvement as shown in Figure 3 (right). These results show that Last-loop provides a favorable balance between reconstruction accuracy and runtime efficiency.

Rotation. To examine whether rotation further improves accuracy, Table 3 compares direct and residual quantization (with LS scaling and the Last-loop policy) with and without rotation. We evaluate INT2 and INT4 using two recent rotation methods, OSCAR (Zhou et al., 2026) and OptR-H (Yun et al., 2026). At INT2, OSCAR and OptR-H improve direct from 50.6% to 56.0% and 56.6%, respectively. Residual quantization achieves 69.0% without rotation, and combining it with OSCAR or OptR-H further improves accuracy to 74.6% and 74.2%, respectively. The same trend holds at INT4, demonstrating the complementary benefits of residual quantization and rotation. The gains with both OSCAR and OptR-H indicate that the benefit of combining residual quantization with rotation is not specific to a single rotation method.

Table 4: Loop-wise precision allocation ablations. MATH500 accuracy (%) on Ouro-1.4B and effective KV bitwidth b_{eff} . We compare direct quantization and residual quantization with and without OptR-H rotation. Residual quantization uses LS scaling and the Last-loop policy. *Uniform* applies the same precision to all loops, *Descending* uses [4, 2, 2, 2], and *Ascending* uses [2, 2, 2, 4], allocating the higher precision to the first and last loop, respectively. Group size g shown in the table applies only to INT2, while INT4 uses group size 32. Blue shading marks our final INT2/4 precision configuration. BF16 accuracy is 75.6%.

Config	g	Direct Quantization			Residual Quantization		
		b_{eff}	w/o Rotation	w/ Rotation	b_{eff}	w/o Rotation	w/ Rotation
<i>Uniform</i>							
[4, 4, 4, 4]	–	4.5	73.60	74.60	4.6	75.60	76.20
[2, 2, 2, 2]	32	2.5	24.40	33.20	2.6	61.20	69.40
[2, 2, 2, 2]	16	3.0	50.60	56.60	3.1	69.00	74.20
<i>Descending</i>							
[4, 2, 2, 2]	32	3.0	25.40	38.60	3.1	62.00	72.40
[4, 2, 2, 2]	16	3.4	56.60	64.20	3.5	70.00	74.20
<i>Ascending</i>							
[2, 2, 2, 4]	32	3.0	32.20	49.20	3.1	73.00	77.60
[2, 2, 2, 4]	16	3.4	60.80	57.60	3.5	76.40	74.00

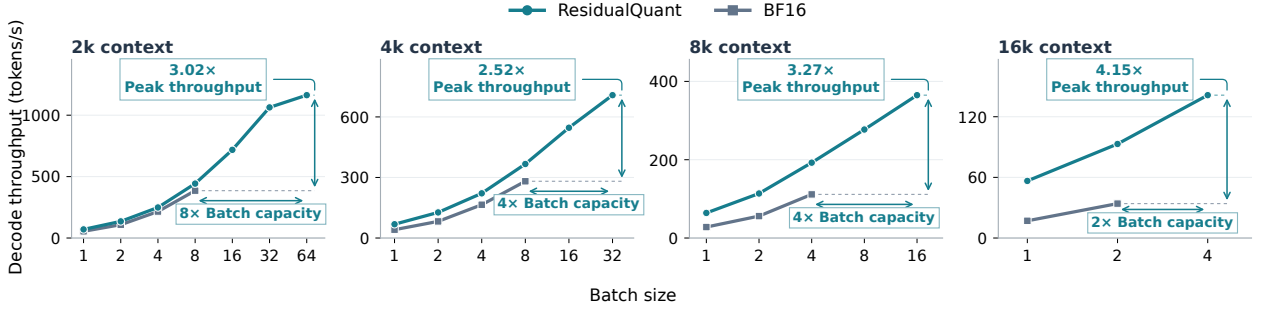


Figure 5: Decode throughput across batch sizes and context lengths. RESIDUALQUANT with group size $g = 32$ in every loop and BF16 on Ouro-1.4B using an RTX 5090, at 2k, 4k, 8k, and 16k context. Vertical and horizontal arrows indicate gains in peak throughput and the largest feasible tested power-of-two batch, respectively.

Loop-wise precision allocation. Finally, we examine how accuracy depends on loop-wise precision allocation. Table 4 compares uniform precision with *Descending* [4, 2, 2, 2] and *Ascending* [2, 2, 2, 4] schedules. Under Last-loop residual quantization, Ascending allocates the higher precision to the shared anchor, whereas Descending allocates it to a residual loop. Ascending generally outperforms Descending with residual quantization. In particular, at $g = 32$, Ascending achieves 73.0% versus 62.0% without rotation and 77.6% versus 72.4% with rotation, supporting the benefit of allocating higher precision to the shared anchor. Moreover, at the same effective bitwidth ($b_{\text{eff}} = 3.1$), Ascending with $g = 32$ outperforms uniform INT2 with $g = 16$ by 4.0 and 3.4 percentage points without and with rotation, respectively, demonstrating a better accuracy–memory trade-off.

5.3. Throughput Improvement

KV compression can improve throughput not only by accelerating a fixed batch but also by allowing more requests to run together on one GPU. Figure 5 extends the comparison in Figure 1 (right) to 2k, 4k, 8k, and 16k contexts. The comparison uses Ouro-1.4B on an RTX 5090 with RESIDUALQUANT (group size $g = 32$ in every loop) and BF16 vLLM FlashAttention, doubling batch sizes from 1 to 128 and measuring feasible configurations. Each run generates 128 tokens. Additional results with INT2 group size $g = 16$ are provided in Appendix J. Reported throughput excludes prefill and is averaged over three runs after one warm-up.

At fixed batch sizes, RESIDUALQUANT improves decode throughput at both context lengths. At 8k context and batch size 4, throughput increases from 111.4 to 192.5 tokens/s (1.73 $\times$). At 16k context and batch size 2, it increases from 34.0 to 93.1 tokens/s (2.73 $\times$). These gains are consistent with the reduced KV cache traffic enabled by compression.

Table 5: Extension of residual quantization to W4A4 activation quantization. Accuracy or pass@1 (%) when applying residual quantization with LS scaling to W4A4 activation quantization, with KV caches kept in BF16. We consider three baselines: Naive round-to-nearest quantization, FlatQuant (Sun et al., 2025), and LoopQ (Fang et al., 2026). For each baseline, colored parentheses report the absolute accuracy change from applying residual quantization. Details are provided in Appendix I.

Method	GSM8K	MATH500	HumanEval	MBPP
Ouro-1.4B (4 loops)				
BF16	78.77	75.00	71.95	74.87
Naive	49.51	53.00	62.20	65.08
+ Residual Quantization	56.86 (+7.35)	61.00 (+8.00)	62.20 (+0.00)	68.52 (+3.44)
FlatQuant	68.31	67.40	67.68	69.58
+ Residual Quantization	70.20 (+1.89)	68.80 (+1.40)	70.12 (+2.44)	71.69 (+2.11)
LoopQ	65.88	64.40	64.63	66.14
+ Residual Quantization	68.31 (+2.43)	64.80 (+0.40)	67.68 (+3.05)	70.90 (+4.76)
Huginn-3.5B (32 loops)				
BF16	42.46	12.40	23.17	41.01
Naive	33.36	9.40	20.12	36.77
+ Residual Quantization	39.12 (+5.76)	12.80 (+3.40)	23.17 (+3.05)	39.68 (+2.91)
FlatQuant	33.21	9.20	18.29	39.15
+ Residual Quantization	38.06 (+4.85)	12.60 (+3.40)	28.05 (+9.76)	37.57 (-1.58)
LoopQ	35.03	11.20	18.90	38.89
+ Residual Quantization	41.02 (+5.99)	12.60 (+1.40)	26.22 (+7.32)	38.89 (+0.00)

The smaller KV footprint additionally allows larger batches. At 8k context, the largest feasible tested power-of-two batch increases from 4 for BF16 to 16 for RESIDUALQUANT, raising throughput further to 364.9 tokens/s and yielding a $3.27\times$ peak-throughput gain over BF16. At 16k context, the corresponding batch increases from 2 to 4, raising throughput to 141.3 tokens/s and yielding a $4.15\times$ peak-throughput gain. Thus, faster decoding at a fixed batch and increased batch capacity jointly improve peak throughput.

5.4. Extension to Activation Quantization

The preceding experiments show that inter-loop similarity can be exploited to compress KV states through low-bit residual quantization. Here, we investigate whether the same principle extends beyond KV storage to activation quantization, which enables low-precision computation in projection layers. Because looped Transformers repeatedly apply the same projections across loops, activations from one loop can also provide a useful reference for the corresponding projection inputs in another loop. We therefore incorporate residual quantization into three W4A4 baselines: Naive, FlatQuant (Sun et al., 2025), an established weight-activation quantization method, and LoopQ (Fang et al., 2026), which is specifically designed for looped Transformers. Naive applies round-to-nearest quantization to weights and activations with group size $g = 32$ without calibration. Implementation and calibration details for FlatQuant and LoopQ are provided in Appendix I.

The baseline methods quantize the full activation independently at each loop. In contrast, our residual quantization uses the Previous-loop policy with LS scaling: we quantize $X_r - \alpha_r X_{r-1}$, where X_r denotes the activation at loop r , apply the low-precision projection to this quantized residual instead. The first loop activation is quantized directly. We use Previous-loop rather than Last-loop because activations are consumed immediately within each loop. Weights and activations remain INT4 and KV caches remain BF16 to isolate the effect of residual quantization on activations.

Table 5 shows that applying residual quantization with LS scaling improves Ouro-1.4B’s GSM8K accuracy from 49.51% to 56.86% for Naive, from 68.31% to 70.20% for FlatQuant, and from 65.88% to 68.31% for LoopQ. On Huginn-3.5B, Naive and FlatQuant improve by 5.76 and 4.85%, respectively. Accuracy improves in nearly all evaluated method-benchmark combinations across both models. These results show that inter-loop residual quantization extends beyond KV compression and can consistently improve existing low-bit activation quantization methods as well.

6. Related Work

KV Cache Quantization. Reducing KV precision saves memory but makes quantization sensitive to uneven distributions and outliers. KIVI and KVQuant address this through quantization tailored to key and value distributions (Liu et al., 2024; Hooper et al., 2024). Rotation-based methods, including QuaRot, SpinQuant, and RotateKV, redistribute outliers across channels to ease low-bit quantization (Ashkboos et al., 2024; Liu et al., 2025; Su et al., 2025). Extending this direction to INT2, OSCAR and OptR optimize rotations with attention-aware objectives, but retain attention sinks and recent KV in BF16 (Zhou et al., 2026; Yun et al., 2026). RESIDUALQUANT exploits inter-loop redundancy to combine INT2 residuals with INT4 anchors, without a separate BF16 buffer for recent past tokens.

Looped Transformers. Reusing parameters across depth enables additional computation without a proportional increase in model size, as explored through cross-layer sharing and equilibrium formulations (Lan et al., 2020; Bai et al., 2019). Adaptive computation further allocates this work according to the input, as in ACT, PonderNet, and Universal Transformers (Graves, 2016; Banino et al., 2021; Dehghani et al., 2019). Recursive architectures extend these ideas through shared blocks, adaptive recursion, and memory of earlier loop states in RRT, MoR, and RecurTrace (Bae et al., 2025a,b; Wang et al., 2026b). Such repeated computation supports length generalization and latent reasoning without intermediate token generation (Fan et al., 2025; Saunshi et al., 2025), with HRM, TRM, and Looped Flows exploring iterative refinement of internal states (Wang et al., 2025; Jolicoeur-Martineau, 2025; Suleymanzade et al., 2026). To distinguish these benefits from simply spending more compute, SMELT studies sparse MoE looping under matched FLOPs, parameter, and KV-cache budgets, demonstrating improved compute efficiency (Wang et al., 2026a).

Optimizing Looped Transformers. Parameter sharing does not eliminate the KV storage and sequential execution costs of additional loops. LoopSpec (Cho et al., 2026) accelerates decoding by drafting tokens from early recurrent states and overlapping draft generation with target verification through pipelined self-speculative decoding. To reduce these costs, MoR selectively caches active tokens and offers first-recursion KV reuse (Bae et al., 2025b). PLT combines first-loop KV sharing with gated sliding-window attention and inter-loop parallelism (Wu et al., 2025). MELT makes cache size independent of loop depth by updating one shared cache per layer through learned gates (Vendrell et al., 2026). Quantization offers a complementary direction: LoopQ addresses inter-loop distribution shifts and error accumulation in W4A4 post-training quantization (Fang et al., 2026), and our residual quantization further improves accuracy when combined with LoopQ and other W4A4 methods (Section 5.4). For KV compression, RESIDUALQUANT preserves loop-specific states without model retraining or pretrained-weight updates, while calibrating OptR-H rotations separately. Concurrent FlashLoop (Yang and Liu, 2026) combines lazy KV updates and quantization motivated by a similar inter-loop observation. A detailed comparison is provided in Appendix K. RESIDUALQUANT systematically studies this design space, identifying Last-loop references and LS scaling (Section 3.2), shared rotations (Section 3.3), and loop-wise mixed precision (Section 3.4) as key to compression, accuracy, and inference efficiency.

7. Conclusion

We introduced RESIDUALQUANT, which uses similarities between loops to compress the KV caches of looped Transformers. A shared Last-loop INT4 anchor predicts each loop’s KV states through LS scaling, while shared OptR-H rotations make the remaining residuals suitable for INT2 quantization. This design preserves information specific to each loop and allows efficient reconstruction during attention. Across mathematical reasoning and code generation benchmarks, RESIDUALQUANT improves the accuracy–memory tradeoff on both Ouro-1.4B and Huginn-3.5B. On Ouro-1.4B with group size $g = 32$, it reduces logical KV storage by 80.7% and achieves 77.6% MATH500 accuracy, compared with 75.6% for BF16. Our attention kernels and vLLM integration turn these memory savings into faster decoding and larger batches. At 8k context on an RTX 5090, RESIDUALQUANT supports $4\times$ the largest feasible tested power-of-two batch and delivers $3.27\times$ the peak decode throughput of BF16. The same residual prediction principle also improves W4A4 accuracy when combined with Naive, FlatQuant, and LoopQ in most evaluated settings, showing that inter-loop predictability benefits both KV compression and activation quantization.

References

- Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. In *NeurIPS*, 2024.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program Synthesis with Large Language Models. *arXiv:2108.07732*, 2021.
- Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. In *ICLR*, 2025a.
- Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. In *NeurIPS*, 2025b.
- Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep Equilibrium Models. In *NeurIPS*, 2019.
- Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to Ponder. In *8th ICML Workshop on Automated Machine Learning*, 2021.
- Yash Bhalgat, Jinwon Lee, Markus Nagel, Tijmen Blankevoort, and Nojun Kwak. LSQ+: Improving Low-Bit Quantization Through Learnable Offsets and Better Initialization. In *CVPR*, 2020. Efficient Deep Learning for Computer Vision Workshop.
- Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. QuIP: 2-Bit Quantization of Large Language Models With Guarantees. In *NeurIPS*, 2023.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*, 2021.
- SangLyul Cho, Langqing Cui, Sehoon Kim, Dongsu Han, and Insu Han. LoopSpec: Pipelined Self-Speculative Decoding for Looped Transformers. *arXiv:2609.17184*, 2026.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168*, 2021.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal Transformers. In *ICLR*, 2019.
- Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped Transformers for Length Generalization. In *ICLR*, 2025.
- Rui Fang, Hsi-Wen Chen, and Ming-Syan Chen. LoopQ: Quantization for Recursive Transformers. *arXiv:2605.16343*, 2026.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. In *NeurIPS*, 2025.
- Alex Graves. Adaptive Computation Time for Recurrent Neural Networks. *arXiv:1603.08983*, 2016.

- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. In *NeurIPS*, 2024.
- Alexia Jolicoeur-Martineau. Less is More: Recursive Reasoning with Tiny Networks. *arXiv:2510.04871*, 2025.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. In *ICLR*, 2020.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s Verify Step by Step. In *ICLR*, 2024.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *NeurIPS*, 2023.
- Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. SpinQuant: LLM Quantization with Learned Rotations. In *ICLR*, 2025.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *ICML*, 2024.
- Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with Latent Thoughts: On the Power of Looped Transformers. In *ICLR*, 2025.
- Zunhai Su, Hanyu Wei, Zhe Chen, Wang Shen, Linge Li, Huangqi Yu, and Kehong Yuan. RotateKV: Accurate and Robust 2-Bit KV Cache Quantization for LLMs via Outlier-Aware Adaptive Rotations. In *IJCAI*, 2025.
- Ayhan Suleymanzade, Chanhyuk Lee, Floor Eijkelboom, Nicholas M. Boffi, İsmail İlkan Ceylan, and Jinwoo Kim. Thinking with Looped Flows. *arXiv:2609.11801*, 2026.
- Yuxuan Sun, Ruikang Liu, Haoli Bai, Han Bao, Kang Zhao, Yuening Li, Jiaxin Hu, Xianzhi Yu, Lu Hou, Chun Yuan, Xin Jiang, Wulong Liu, and Jun Yao. FlatQuant: Flatness Matters for LLM Quantization. In *ICML*, 2025.
- Victor Conchello Vendrell, Arnau Padrés Masdemont, Niccolò Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models. *arXiv:2605.07721*, 2026.
- Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical Reasoning Model. *arXiv:2506.21734*, 2025.
- Shaowen Wang, Ge Zhang, Kairong Luo, Yuhao Wu, Shaofan Liu, Jiaheng Liu, Wenhao Huang, Shen Yan, and Jian Li. SMELT: Scaling Laws for Compute-Matched MoE Looped Transformers. *arXiv:2609.01343*, 2026a.
- Yuxiang Wang, Kunyu Feng, Yingda Shen, Haoning Xu, Junyu Wang, and Zhizheng Wu. RecurTrace: Adaptive Latent Reasoning with Loop-Time Memory. *arXiv:2609.03379*, 2026b.
- Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv:2510.24824*, 2025.
- Wanqi Yang and Shiwei Liu. FlashLoop: Fast and Memory-Efficient Looped Transformers via Lazy Updates. *arXiv:2609.29812*, 2026.
- Vincent-Daniel Yun, Woosang Lim, Minsoo Cheong, Sunwoo Lee, Murali Annavaram, Sai Praneeth Karimireddy, and Sungjoo Yoo. Output-Aware Rotation for INT2 KV-Cache Quantization. *arXiv:2608.02691*, 2026.

Zhongzhu Zhou, Donglin Zhuang, Jisen Li, Ziyan Chen, Shuaiwen Leon Song, Ben Athiwaratkun, and Xiaoxia Wu. OSCAR: Offline Spectral Covariance-Aware Rotation for 2-bit KV Cache Quantization. *arXiv:2605.17757*, 2026.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling Latent Reasoning via Looped Language Models. *arXiv:2510.25741*, 2025.

A. Experimental Setup Details

A.1. Models and Hardware

Ouro-1.4B uses the ByteDance/Ouro-1.4B checkpoint, and Huginn-3.5B uses `tomg-group-umd/huginn-0125`. Ouro-1.4B experiments run on an NVIDIA RTX 5090 with 32 GB of memory, and Huginn-3.5B experiments run on an NVIDIA H100 with 80 GB of memory. Each evaluation uses a single GPU, BF16 weights, a compressed KV backend integrated into vLLM, and decode CUDA Graphs. Ouro-1.4B executes 4 loops and Huginn-3.5B 32 loops, with residual references confined to each group of four loops.

A.2. Benchmarks and Prompts

Table 6: Prompt formats and maximum generated tokens. Ouro-1.4B uses plain-text prompts, while Huginn-3.5B uses its tokenizer’s chat template.

Model	Benchmark	Shots	Prompt format	Max tokens
Ouro-1.4B	GSM8K	3	GSM8K CoT	1,024
	MATH500	0	CoT, boxed answer	2,048
	HumanEval	0	EvalPlus-style	1,024
	MBPP	0	EvalPlus-style	2,048
Huginn-3.5B	GSM8K	8	GSM8K CoT	1,024
	MATH500	4	Adapted Minerva	2,048
	HumanEval	0	EvalPlus-style	1,024
	MBPP	0	EvalPlus-style	2,048

The evaluations cover 1,319 GSM8K problems, 500 MATH500 problems, 164 HumanEval problems, and 378 MBPP problems. For mathematical reasoning, GSM8K is scored by exact match on extracted final answers, and MATH500 is scored using Math-Verify. Code evaluation uses the curated tasks and evaluation tools from EvalPlus (Liu et al., 2023), while the reported metric is base pass@1 on the original tests. All methods use the same model-specific prompts and stopping conditions, generating one response per problem with greedy decoding. Table 6 summarizes prompt formats and generation limits.

Ouro-1.4B uses no chat template for any benchmark. Huginn-3.5B uses *You are a helpful assistant that can assist users with mathematical reasoning.* as the GSM8K system instruction and no separate system instruction for MATH500. The custom Ouro-1.4B MATH500 prompt requests step-by-step reasoning. MATH500 prompts request a final answer inside `\boxed{\}`, with Huginn-3.5B additionally instructed to use the `Final Answer:` format. Code prompts request Python code and provide an assistant response prefix with an open code fence. Generated outputs are cleaned before running the tests.

A.3. Effective KV Bitwidth

We measure KV-cache storage using the effective bitwidth b_{eff} , defined as the total number of bits required to store token-dependent quantized KV codes and their associated metadata, divided by the number of original KV elements. For a group of L loops with equal KV sizes,

$$b_{\text{eff}} = \frac{1}{L} \sum_{r=1}^L \left(b_r + \frac{16}{g_r} \right) + c_{\alpha}, \quad (\text{A.1})$$

where b_r and g_r denote the quantization precision and group size, respectively, for loop r . The term $16/g_r$ accounts for the two FP8 metadata values—a scale and an offset—stored for each quantization group.

For residual quantization with LS scaling, we additionally store one BF16 scaling coefficient for each residual loop, separately for keys and values. Its amortized storage cost is

$$c_{\alpha} = \frac{16(L-1)}{LD}, \quad (\text{A.2})$$

where D is the head dimension. For direct quantization and OptR-H, which do not require these coefficients, $c_\alpha = 0$.

For Ouro-1.4B, with $L = 4$ and $D = 128$, LS coefficients contribute 0.094 additional bits per KV element. Consequently, the four precision settings used in our main comparison have effective bitwidths of 4.5, 3.0, 3.0, and 2.5 bits for direct quantization and OptR-H, corresponding respectively to INT4 with $g = 32$, INT2/4 with $g = 32$, INT2 with $g = 16$, and INT2 with $g = 32$. For LS-based methods, these become approximately 4.6, 3.1, 3.1, and 2.6 bits due to the additional LS coefficients. We include integer codes, scales, offsets, and LS coefficients in b_{eff} , but exclude fixed rotations and means, as well as runtime padding and temporary buffers.

B. Analysis Details

Key prediction MSE (Figure 3). The left and middle panels of Figure 3 use 512 GSM8K training examples, averaging per-example statistics with equal weights. The right panel follows the generation and timing protocol in Section 5.3. For the middle panel, using one-based loop indices, let $K_{i,r,j}$ be the original key vector for problem i , loop $r \in \{1, 2, 3\}$, and layer–token–head position j . The Last-loop reference is the reconstructed anchor $R_{i,j} = \hat{K}_{i,4,j}$. We measure

$$\text{MSE}_{i,r}^{\text{pred}} = \frac{1}{N_{i,r}} \sum_j \|K_{i,r,j} - \alpha_{i,r,j} R_{i,j}\|_2^2, \quad (\text{B.1})$$

where $N_{i,r}$ counts all key elements included in the sum. We compare $\alpha_{i,r,j} = 1$ with LS scaling, $\alpha_{i,r,j} = \langle K_{i,r,j}, R_{i,j} \rangle / \|R_{i,j}\|_2^2$, using the same reconstructed anchor under uniform INT2 or INT4 with group size $g = 32$ and no rotation. This measures the residual before residual quantization, not the final key reconstruction error. We average MSE equally across the 512 problems.

Key reconstruction NMSE (Table 1). For Table 1, we collect BF16 post-RoPE keys using teacher forcing on the full prompts and gold solutions. Both reference directions use the same BF16 states and the packed cache writer and reader, without feedback from quantized attention. We compute $\|\hat{K} - K\|_F^2 / \max(\|K\|_F^2, 10^{-30})$ per problem and loop over all tokens, layers, heads, and channels, then average equally across problems and multiply by 10^3 .

C. Key Distributions Relative to the Last-loop Anchor

Using the same input as Figure 2, Figure 6 compares the keys from loops 1, 2, and 3 with the loop 4 anchor and their differences $K_r - K_4$. The data come from physical layer 3 and KV head 2 of Ouro-1.4B, with both indices starting at 1. Post-RoPE keys are collected during BF16 execution on a 573-token input comprising the fourth GSM8K training example and the preceding three examples as demonstrations. All tokens and 128 channels are shown, with token and channel indices starting at 0. Panels display absolute values on shared height and color scales. The differences are $K_r - K_4$ before LS adjustment or quantization.

D. KV Reconstruction for Four Reference Choice Configurations

Let $X_1, \dots, X_4$ denote Ouro-1.4B’s KV states at a fixed layer, token, and head. Residual quantization with LS scaling and shared OptR-H rotations reconstructs states in the rotated space $Z_r = UX_r$, yielding $\hat{X}_r = U^\top \hat{Z}_r$ in the original space. K and V use separate matrices U . Residual quantization without rotation corresponds to $U = I$.

Let a denote the anchor loop and $p(r)$ the reference for loop r . The reconstructed anchor and residuals are

$$A_a = Q(Z_a - \mu_a) + \mu_a, \quad \hat{E}_r = Q(Z_r - \alpha_r \hat{Z}_{p(r)} - \mu_r) + \mu_r \quad (\text{D.1})$$

Here, Q denotes quantization followed by reconstruction at the precision assigned to the corresponding loop, and α_r is the stored LS coefficient. The fixed key means μ_a and μ_r correspond to the rotated anchor and residuals, respectively. They are zero for V and for residual quantization without rotation. Each loop is reconstructed after recovering A_a and $\hat{E}_r$ from the cached codes, scales, and offsets.

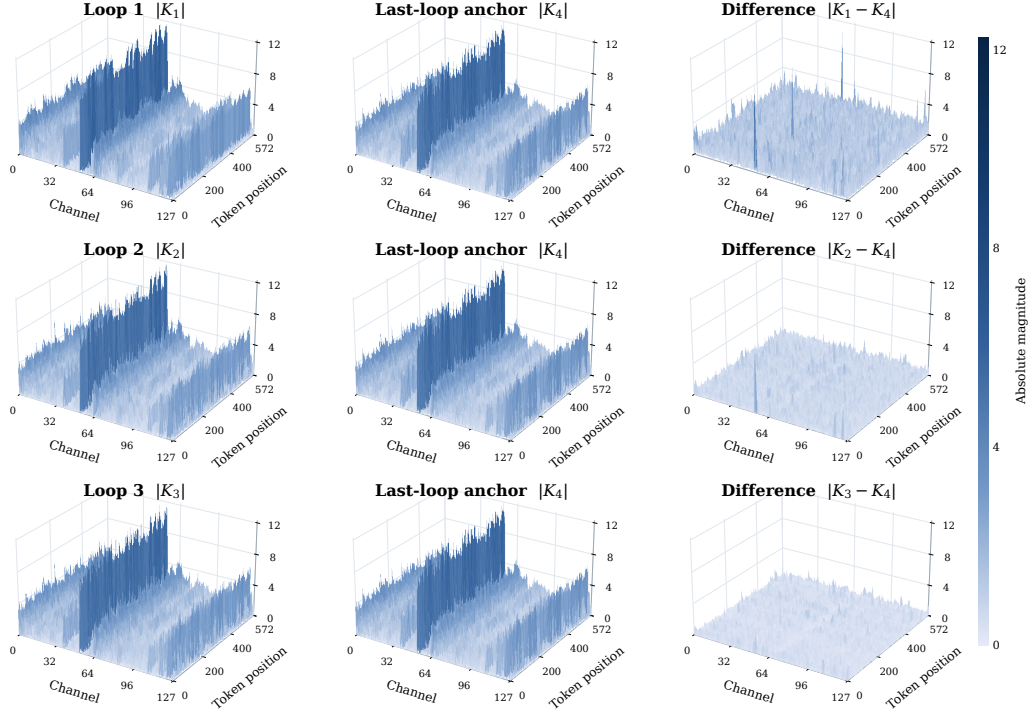


Figure 6: Key distributions relative to the last-loop anchor. Rows correspond to loops 1, 2, and 3. Columns show $|K_r|$, $|K_4|$, and $|K_r - K_4|$ on shared scales.

Previous-loop. Loop 1 is the anchor, with $p(r) = r - 1$. Reconstruction proceeds in the order $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$.

$$\hat{Z}_1 = A_1, \quad \hat{Z}_2 = \alpha_2 A_1 + \hat{E}_2, \quad \hat{Z}_3 = \alpha_3 \hat{Z}_2 + \hat{E}_3, \quad \hat{Z}_4 = \alpha_4 \hat{Z}_3 + \hat{E}_4. \quad (\text{D.2})$$

Reconstructing loop 4 requires the anchor followed by the residuals of loops 2, 3, and 4.

Next-loop. Loop 4 is the anchor, with $p(r) = r + 1$. Reconstruction proceeds in the order $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$.

$$\hat{Z}_4 = A_4, \quad \hat{Z}_3 = \alpha_3 A_4 + \hat{E}_3, \quad \hat{Z}_2 = \alpha_2 \hat{Z}_3 + \hat{E}_2, \quad \hat{Z}_1 = \alpha_1 \hat{Z}_2 + \hat{E}_1. \quad (\text{D.3})$$

Reconstructing loop 1 requires the anchor followed by the residuals of loops 3, 2, and 1. This order describes reconstruction dependencies. The model still executes loops in the order $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$.

First-loop. Loops 2, 3, and 4 directly reference loop 1, giving $p(r) = 1$.

$$\hat{Z}_1 = A_1, \quad \hat{Z}_2 = \alpha_2 A_1 + \hat{E}_2, \quad \hat{Z}_3 = \alpha_3 A_1 + \hat{E}_3, \quad \hat{Z}_4 = \alpha_4 A_1 + \hat{E}_4. \quad (\text{D.4})$$

Each residual loop is reconstructed from A_1 and its own residual.

Last-loop. Loops 1, 2, and 3 directly reference loop 4, giving $p(r) = 4$.

$$\hat{Z}_4 = A_4, \quad \hat{Z}_1 = \alpha_1 A_4 + \hat{E}_1, \quad \hat{Z}_2 = \alpha_2 A_4 + \hat{E}_2, \quad \hat{Z}_3 = \alpha_3 A_4 + \hat{E}_3. \quad (\text{D.5})$$

Each residual loop is reconstructed from A_4 and its own residual.

Thus, under First-loop and Last-loop, reconstructing any residual loop requires only two KV-cache reads: the shared anchor and that loop’s residual. In contrast, Previous-loop and Next-loop require loading a chain of intermediate residuals, with the number of reads increasing with the distance from the anchor and reaching four reads for the farthest loop in a four-loop model.

Logical reference count. For a group of L loops, consider the number of compressed components required to reconstruct every loop’s KV. Neighboring-loop reconstruction uses $1, 2, \dots, L$ components depending on distance from the anchor, for a total of $L(L+1)/2$ accesses. Shared-anchor reconstruction uses one component for the anchor and two for each of the remaining $L-1$ loops, totaling $2L-1$. The ratio is therefore $L(L+1)/[2(2L-1)]$, giving $10/7 \approx 1.43$ for $L=4$ and $528/63 \approx 8.38$ for $L=32$. These are logical component accesses, not byte counts accounting for bitwidth, metadata size, cache hits, or actual HBM traffic. Huginn-3.5B’s eight independent 4-loop groups require 80 and 56 accesses, respectively, retaining the ratio of 1.43.

E. Huginn-3.5B Anchor Interval

Table 7: Huginn-3.5B GSM8K accuracy (%), 1,319 questions) and effective KV bitwidth b_{eff} across anchor intervals. Each group uses one final INT4 anchor. $2^{\times n}$ denotes n consecutive INT2 loops. Group size g applies to INT2, while INT4 uses $g=32$. Both residual quantization variants use LS scaling and the Last-loop policy. Effective bitwidth includes quantized values, scales, offsets, and LS coefficients, following Appendix A.3, and is the same with and without OptR-H.

Schedule	g	b_{eff}	w/o OptR-H	w/ OptR-H
BF16	–	16		42.84
$8 \times [2, 2, 2, 4]$	16	3.5	41.70	42.38
	32	3.1	43.14	42.61
$4 \times [2^{\times 7}, 4]$	16	3.3	41.55	42.53
	32	2.9	41.93	42.15
$2 \times [2^{\times 15}, 4]$	16	3.3	41.93	42.68
	32	2.8	42.61	42.30

The comparison keeps 32 total loops and varies schedules that use the final loop of each group as an INT4 anchor. Separate 96×96 rotations for K and V are learned for each group, layer, and head and shared within the group.

Although we use four-loop groups by default, grouping eight or sixteen loops retains comparable GSM8K accuracy with OptR-H while reducing the number of INT4 anchors. Increasing the group length from four to sixteen loops reduces effective bitwidth from 3.5 to 3.3 at $g=16$ and from 3.1 to 2.8 at $g=32$. The sixteen-loop configurations reduce logical KV storage by 79.7% and 82.6% relative to BF16, respectively. Accuracy changes from 42.38% to 42.68% and from 42.61% to 42.30%, respectively. Larger loop groups thus offer an additional option for reducing KV storage while retaining similar accuracy.

F. Full Benchmark Results

Table 8: Math accuracy and code pass@1 (%) for Ouro-1.4B and Huginn-3.5B. Parentheses report logical b_{eff} . Huginn-3.5B repeats each four-loop schedule eight times. INT2 group size is shown in the table, while INT4 uses $g = 32$. Residual quantization uses Last-loop references and LS scaling. Detailed evaluation settings are in Appendix A. Figure 1 uses MATH500 results with group size $g = 32$.

		Ouro-1.4B (4 loops)					Huginn-3.5B (32 loops, 8×Schedule)				
Benchmark	Schedule	g	Direct	OptR-H	Residual	Residual	Direct	OptR-H	Residual	Residual	
					Quantization	Quantization			+ OptR-H	Quantization	+ OptR-H
Math											
GSM8K	BF16	–	79.23 (16.0)				42.84 (16.0)				
	[2, 2, 2, 2]	32	16.07 (2.5)	41.70 (2.5)	64.90 (2.6)	69.75 (2.6)	17.74 (2.5)	24.72 (2.5)	33.97 (2.6)	37.30 (2.6)	
	[2, 2, 2, 2]	16	42.76 (3.0)	56.86 (3.0)	71.95 (3.1)	74.45 (3.1)	21.15 (3.0)	29.64 (3.0)	35.41 (3.1)	37.98 (3.1)	
	[2, 2, 2, 4]	32	33.59 (3.0)	52.31 (3.0)	77.41 (3.1)	77.86 (3.1)	25.09 (3.0)	31.24 (3.0)	43.14 (3.1)	42.61 (3.1)	
	[2, 2, 2, 4]	16	52.54 (3.4)	64.52 (3.4)	76.95 (3.5)	78.24 (3.5)	26.54 (3.4)	34.12 (3.4)	41.70 (3.5)	42.38 (3.5)	
	[4, 4, 4, 4]	–	75.36 (4.5)	75.28 (4.5)	78.17 (4.6)	77.71 (4.6)	40.26 (4.5)	40.64 (4.5)	42.46 (4.6)	42.91 (4.6)	
MATH500	BF16	–	75.60 (16.0)				13.20 (16.0)				
	[2, 2, 2, 2]	32	24.40 (2.5)	33.20 (2.5)	61.20 (2.6)	69.40 (2.6)	7.80 (2.5)	8.80 (2.5)	12.60 (2.6)	13.80 (2.6)	
	[2, 2, 2, 2]	16	50.60 (3.0)	56.60 (3.0)	69.00 (3.1)	74.20 (3.1)	9.40 (3.0)	10.80 (3.0)	12.80 (3.1)	14.60 (3.1)	
	[2, 2, 2, 4]	32	32.20 (3.0)	49.20 (3.0)	73.00 (3.1)	77.60 (3.1)	11.20 (3.0)	10.20 (3.0)	14.20 (3.1)	15.20 (3.1)	
	[2, 2, 2, 4]	16	60.80 (3.4)	57.60 (3.4)	76.40 (3.5)	74.00 (3.5)	10.40 (3.4)	11.40 (3.4)	14.60 (3.5)	13.60 (3.5)	
	[4, 4, 4, 4]	–	73.60 (4.5)	74.60 (4.5)	75.60 (4.6)	76.20 (4.6)	15.20 (4.5)	14.20 (4.5)	13.60 (4.6)	15.20 (4.6)	
Code											
HumanEval	BF16	–	70.12 (16.0)				23.17 (16.0)				
	[2, 2, 2, 2]	32	32.32 (2.5)	57.32 (2.5)	57.93 (2.6)	67.07 (2.6)	17.07 (2.5)	20.73 (2.5)	24.39 (2.6)	23.78 (2.6)	
	[2, 2, 2, 2]	16	59.15 (3.0)	62.80 (3.0)	65.85 (3.1)	72.56 (3.1)	16.46 (3.0)	15.85 (3.0)	26.22 (3.1)	24.39 (3.1)	
	[2, 2, 2, 4]	32	46.95 (3.0)	67.07 (3.0)	69.51 (3.1)	70.73 (3.1)	17.68 (3.0)	19.51 (3.0)	23.17 (3.1)	25.00 (3.1)	
	[2, 2, 2, 4]	16	63.41 (3.4)	71.34 (3.4)	68.29 (3.5)	71.95 (3.5)	17.07 (3.4)	17.07 (3.4)	23.17 (3.5)	23.17 (3.5)	
	[4, 4, 4, 4]	–	73.17 (4.5)	70.12 (4.5)	71.95 (4.6)	71.95 (4.6)	23.17 (4.5)	23.17 (4.5)	23.17 (4.6)	23.17 (4.6)	
MBPP	BF16	–	74.34 (16.0)				41.27 (16.0)				
	[2, 2, 2, 2]	32	44.44 (2.5)	64.81 (2.5)	66.67 (2.6)	71.69 (2.6)	30.42 (2.5)	35.71 (2.5)	39.15 (2.6)	41.27 (2.6)	
	[2, 2, 2, 2]	16	64.81 (3.0)	69.31 (3.0)	71.43 (3.1)	71.96 (3.1)	31.48 (3.0)	37.04 (3.0)	37.30 (3.1)	40.74 (3.1)	
	[2, 2, 2, 4]	32	59.79 (3.0)	67.46 (3.0)	73.02 (3.1)	73.02 (3.1)	33.33 (3.0)	37.04 (3.0)	42.06 (3.1)	40.74 (3.1)	
	[2, 2, 2, 4]	16	69.58 (3.4)	69.84 (3.4)	74.60 (3.5)	73.54 (3.5)	35.45 (3.4)	37.57 (3.4)	40.74 (3.5)	40.21 (3.5)	
	[4, 4, 4, 4]	–	73.54 (4.5)	73.81 (4.5)	74.60 (4.6)	75.13 (4.6)	38.62 (4.5)	40.48 (4.5)	42.86 (4.6)	42.06 (4.6)	

G. Full KV Quantization Ablations

Table 9: Scaling coefficient, reference, and rotation ablations. MATH500 accuracy (%) on Ouro-1.4B with uniform INT2 and INT4. Direct quantization is independent of the reference policy. Residual variants compare different scaling coefficients and reference choices, with OSCAR (Zhou et al., 2026) or OptR-H (Yun et al., 2026) applied to residual quantization with LS scaling. Blue row shading marks the selected Last-loop policy, blue column shading marks residual quantization with LS scaling and OptR-H, and the darker intersections indicate our final configuration. BF16 accuracy is 75.6%.

Reference / Method	No rotation				+ OSCAR		+ OptR-H	
	Direct	Residual	Norm-ratio	LS	Direct	LS	Direct	LS
<i>INT2</i>								
Direct	50.60	–	–	–	56.00	–	56.60	–
Previous-loop	–	63.60	69.00	69.60	–	70.20	–	62.80
Next-loop	–	68.20	68.60	69.20	–	74.60	–	73.20
First-loop	–	38.60	61.40	62.60	–	66.60	–	68.80
Last-loop	–	67.60	70.00	69.00	–	74.60	–	74.20
<i>INT4</i>								
Direct	73.60	–	–	–	75.60	–	74.60	–
Previous-loop	–	73.40	76.40	73.00	–	75.20	–	74.80
Next-loop	–	75.40	75.40	76.20	–	76.40	–	75.80
First-loop	–	73.20	76.60	74.60	–	74.80	–	75.00
Last-loop	–	75.60	73.80	75.60	–	76.00	–	76.20

Table 9 retains the complete comparison underlying Tables 3 and 2, including Norm-ratio scaling and both OSCAR and OptR-H rotations. The direct and rotation-only baselines are independent of the reference choice

and are listed once per precision. For a KV vector X_r and its reconstructed reference R_r , we compare residual quantization with three scaling choices: $\alpha_r = 1$, Norm-ratio scaling with $\alpha_r = \|X_r\|_2 / \|R_r\|_2$, and LS scaling with $\alpha_r = \langle X_r, R_r \rangle / \|R_r\|_2^2$, which minimizes $\|X_r - \alpha_r R_r\|_2^2$. The LS + OSCAR and LS + OptR-H columns apply residual quantization with the same LS coefficient formula in the rotated space. Neither coefficient choice uniformly dominates the other in downstream accuracy.

H. Rotation Calibration and Inference

OSCAR and OptR-H both determine K and V rotations before inference, but use different criteria to obtain them. This section describes how the rotation matrices are calibrated offline and how the resulting fixed matrices are applied during inference.

H.1. Calibration

Let Q, K, V contain calibration tokens as rows for a fixed layer and head, and let $S = \text{softmax}(QK^\top / \sqrt{D})$ denote attention probabilities with a causal mask. The row-vector rotation corresponding to the main text’s column-vector rotation U is $T = U^\top$, applied to X as XT . The rotations T_K and T_V are obtained separately. In RESIDUALQUANT, the anchor and residuals in each four-loop group share the same matrices.

Quantized reference and residual targets. Q/K/V states are collected from the BF16 model, but residual references use quantized and reconstructed KV rather than original KV. With the last loop as anchor, X_4 is first quantized and reconstructed, after which each token’s coefficient $\alpha_r = \langle X_r, \hat{X}_4 \rangle / \|\hat{X}_4\|_2^2$ and residual $E_r = X_r - \alpha_r \hat{X}_4$ are computed. The quantization targets are X_4 for the anchor and E_r for residuals. Rotation-only baselines instead target original KV.

OSCAR. OSCAR (Zhou et al., 2026) constructs rotations from covariance estimates of directions important for QK scores and weighted V outputs. Let Z_V be the V quantization target, either original V or a residual computed with LS scaling, and N_q the number of queries. Our adaptation uses the statistics and rotations

$$C_K = \frac{Q^\top Q}{N_q}, \quad C_V = \frac{(SZ_V)^\top (SZ_V)}{N_q}, \quad C_X = B_X \Lambda_X B_X^\top, \quad T_X = B_X H P_X. \quad (\text{H.1})$$

Here, B_X contains covariance eigenvectors, H is a normalized Hadamard matrix, and P_X is a bit-reversal permutation based on eigenvalue ranks. Eigenvectors align important directions, while Hadamard mixing and permutation redistribute them across groups. Shared rotations aggregate statistics over the corresponding loops using all calibration queries. Attention probabilities S remain those computed from original Q/K, while LS variants use residuals as Z_V . Covariances and eigendecompositions are computed in FP64, without gradient-based rotation learning.

OptR-H. For OptR-H, mathematical reasoning evaluations share rotation matrices calibrated on the GSM8K training set, while code evaluations use matrices calibrated on the MBPP training set after excluding examples whose IDs overlap the evaluation set. Each domain uses 128 fit examples and 128 holdout examples with seed 0.

OptR (Yun et al., 2026) optimizes attention error induced by quantized KV. Our OptR-H implementation maintains orthogonality through $T = T_0 \exp(A - A^\top)$ with Hadamard initialization $T_0 = H$. Let $\hat{S} = \text{softmax}(Q\hat{K}^\top / \sqrt{D})$ and let W_O denote the output projection for the head. The K and V objectives are

$$\mathcal{L}_K = \overline{D}_{\text{KL}}(S \| \hat{S}) + \mathcal{E}((\hat{S} - S)V), \quad \mathcal{L}_V = \mathcal{E}(\hat{S}(\hat{V} - V)), \quad \mathcal{E}(\Delta) = \frac{\|\Delta W_O\|_F^2}{N_q d_{\text{model}}}. \quad (\text{H.2})$$

Here, $\overline{D}_{\text{KL}}$ is the mean query-wise KL divergence. The K rotation is selected first, and V rotation learning then fixes $\hat{S}$ computed from reconstructed K. Output projection error is evaluated per head using $W_O W_O^\top$. Shared matrices minimize the mean loss across loops in the group.

For Ouro-1.4B, rotations and optimizer states use FP32 with Adam at learning rate 0.02 for 80 steps. Each step uses the last 64 queries from two calibration examples. Quantization uses straight-through gradients, while gradients are stopped through reconstructed references. Per-head matrices are selected by holdout loss.

H.2. Inference

After calibration, matrices are fixed and stored in FP16. Ouro-1.4B uses 128×128 rotations, while Huginn-3.5B uses 96×96 rotations matching its head dimension. Rotation multiplication uses FP16 inputs and FP32 accumulation. No further rotation training occurs during token generation.

Quantization and storage. Returning to column-vector notation, let $Z_r = UX_r$. Distinguish quantization into integer codes, $\mathcal{Q}_b$, from reconstruction using scales and offsets, $\mathcal{D}_b$. With fixed key mean μ_r in the rotated space, the last anchor and residuals are stored as

$$c_4 = \mathcal{Q}_4(Z_4 - \mu_4), \quad A = \mathcal{D}_4(c_4) + \mu_4, \quad c_r = \mathcal{Q}_2(Z_r - \alpha_r A - \mu_r) \quad (r = 1, 2, 3). \quad (\text{H.3})$$

For V and OSCAR, $\mu_r = 0$. The cache stores integer codes, group-wise FP8 scales and offsets, and BF16 LS coefficients for residuals. Matrices and fixed means are shared separately from the per-token cache. During decode, the current token participates in attention using its own BF16 KV. Compressed storage of its anchor and three residuals completes once the final loop’s KV is available.

Reconstruction and attention. Rotated KV for past tokens is reconstructed as $\hat{Z}_4 = A$ and $\hat{Z}_r = \alpha_r A + \mathcal{D}_2(c_r) + \mu_r$. Applying the same K rotation to the query gives $q^\top \hat{k}_r = (U_K q)^\top \hat{z}_{K,r}$, avoiding inverse rotation of all keys. Values are inverse-rotated once after weighted accumulation, as $o = U_V^\top \sum_t p_t \hat{z}_{V,r,t}$. The kernel reads compressed KV in tiles and computes anchor and residual contributions without creating a separate cache of fully reconstructed past KV. OSCAR and OptR-H thus supply fixed matrices to the same compressed attention path, allowing comparison of their calibration methods.

I. Activation Residual Quantization

Table 5 compares direct and residual activation quantization with LS scaling at W4A4. Weight and activation codes use INT4, while KV caches remain in BF16. The BF16 baseline in this section is evaluated alongside the activation experiments and is distinct from the KV experiment baseline.

I.1. Baseline Methods and Implementation

All W4A4 configurations use group size $g = 32$ for both weights and activations, with BF16 KV caches.

Naive. Naive applies symmetric round-to-nearest quantization directly to weights and activations without a learned transformation or calibration. Its LS residual variant also requires no calibration.

FlatQuant. FlatQuant (Sun et al., 2025) learns invertible affine transformations to flatten weight and activation distributions before quantization. The corresponding inverse transformation is folded into the weights, preserving the full-precision linear mapping before quantization. Kronecker-factorized matrices reduce transformation cost, while learnable per-channel scaling and weight and activation clipping further control outliers. These parameters are jointly calibrated by minimizing the output reconstruction error of each Transformer block. We use the authors’ transformation, clipping, and calibration modules, adapted to the projection layouts of Ouro-1.4B and Huginn-3.5B. Each projection’s transformation is shared across loops.

LoopQ. LoopQ (Fang et al., 2026) addresses loop-dependent distribution shifts and recursive error accumulation while retaining a shared quantized backbone. Loop-aware Activation Scaling (LAS) assigns activation

ranges to individual loops. Selective Loop-aware Transformation (SLT) introduces loop-dependent transformations only for modules identified as sensitive to sharing by curvature-normalized inter-loop gradient variance. Lightweight transition adapters align each loop’s quantized output before it enters the next loop. Trajectory-aware calibration jointly optimizes these components using full-precision logits, intermediate and final hidden states, and transition-alignment targets along the recurrent computation path.

In the absence of the authors’ code, we reproduced LoopQ to the best of our ability by following the published methodology and matching the reported results across all benchmarks.

1.2. Residual Formulation

Let the row vector x_r denote the current activation for a projection and $\hat{x}_{r-1}$ the reconstructed reference from the previous loop. The first loop directly quantizes the full activation. Subsequent loops compute residuals using a token-wise scalar LS coefficient.

$$\alpha_r = \frac{\langle x_r, \hat{x}_{r-1} \rangle}{\|\hat{x}_{r-1}\|_2^2}, \quad e_r = x_r - \alpha_r \hat{x}_{r-1}, \quad \hat{x}_r = \alpha_r \hat{x}_{r-1} + Q(e_r). \quad (\text{I.1})$$

Here, Q includes the method’s transformation and INT4 quantization and reconstruction. We apply residual quantization with LS scaling to QKV, attention output, MLP gate/up, and down projections.

1.3. Calibration

For both models, FlatQuant direct calibration uses 128 Pile samples of 2,048 tokens each, seed 0, 15 epochs, and batch size 4. Its LS residual variant initializes from the corresponding direct artifact and undergoes a further 15 epochs of calibration through the residual computation path, using the same calibration tokens and batch size. Residuals are formed before the FlatQuant transformation, and the quantized residual is passed to the low-precision projection. Earlier quantized projections provide reconstructed references during residual calibration.

The Ouro-1.4B LoopQ LS residual variant reuses the previously calibrated direct bundle without additional training. For Huginn-3.5B, LoopQ direct calibration uses 1,024 Pile samples of up to 256 tokens each. The recorded calibration recipe specifies three layerwise epochs and a 500-step setting. The archived Ouro-1.4B direct bundle does not include its original calibration run record, so we do not infer that it used the Huginn-3.5B training schedule.

1.4. Evaluation Protocol

Table 5 evaluates all 1,319 GSM8K, 500 MATH500, 164 HumanEval, and 378 MBPP problems. GSM8K uses flexible-extract exact match, with three-shot plain-text prompts for Ouro-1.4B and eight-shot prompts using the authors’ chat template and system instruction for Huginn-3.5B. The generation limit is 1,024 tokens.

Ouro-1.4B MATH500 uses zero-shot prompts, a 2,048-token generation limit, and Math-Verify. Huginn-3.5B MATH500 uses the four-shot `minerva_math500` task with plain-text prompts, no chat template or system instruction, a 2,048-token generation limit, and Minerva exact match. Huginn-3.5B evaluations split the 500 problems into disjoint shards of 167, 167, and 166 problems and aggregate correct-answer counts over all 500 problems. This activation experiment protocol differs from the Huginn-3.5B KV evaluation protocol in Appendix A.

HumanEval and MBPP use zero-shot EvalPlus prompts and report base pass@1 on the original tests, with generation limits of 1,024 and 2,048 tokens, respectively. Model-specific prompts and scoring settings are shared between direct and LS residual variants. BF16 and Naive require no calibration, and all reported activation experiments retain BF16 KV caches.

J. Decode Throughput with INT2 Group Size 16

Figure 7 reports the additional throughput results with group size $g = 16$ for INT2 residuals and $g = 32$ for the INT4 anchor. At 8k context, peak throughput increases from 113.1 to 367.4 tokens/s ($3.25\times$), while the largest tested feasible power-of-two batch increases from 4 to 16.

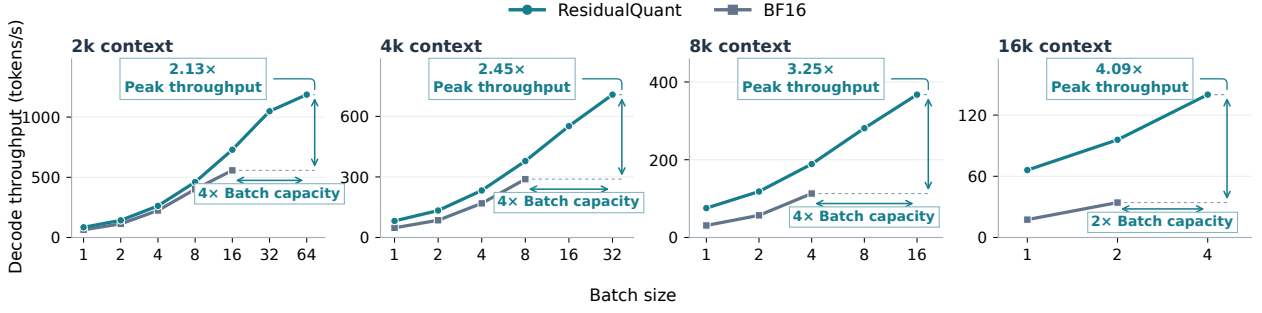


Figure 7: Decode throughput with INT2 group size $g = 16$. Ouro-1.4B on an RTX 5090 at 2k, 4k, 8k, and 16k context. RESIDUALQUANT uses [2, 2, 2, 4], Last-loop references, LS scaling, and shared OptR-H rotations, with group size $g = 16$ for INT2 residuals and $g = 32$ for the INT4 anchor. Each run generates 128 tokens, with throughput measured over the 127 decode steps after the first output token and averaged over three trials following one warm-up. Arrows show peak-throughput gains and ratios of the largest plotted batch sizes relative to BF16.

K. Comparison with FlashLoop

FlashLoop (Yang and Liu, 2026) is concurrent work that combines INT4 KV quantization with sparse token updates and sparse attention for looped Transformer. While its quantization method is motivated by a similar observation of inter-loop redundancy, RESIDUALQUANT improves both quantization accuracy and runtime efficiency, enabling residual KV states to be reduced from INT4 to INT2. Moreover, our quantization method is orthogonal to FlashLoop’s sparse-update and sparse-attention techniques, allowing the two to be combined for further reductions in KV memory and computation while preserving higher accuracy. We examine these aspects below.

Run-time efficiency. FlashLoop performs residual quantization using a Previous-loop reference, where each loop is reconstructed from its predecessor. This introduces a chain of reconstruction dependencies: recovering a later loop requires loading the anchor together with multiple intermediate residuals (Section 3.2 and Appendix D). In contrast, our Last-loop policy reconstructs every residual loop directly from the final-loop anchor, reducing the required KV cache reads. As shown in Figure 3, under the same quantization setting, the Next-loop policy can only achieve 62% of the decode throughput of the Last-loop policy due to larger memory traffic.

Low-precision accuracy. We further find that both the reference choice and LS scaling are important for preserving accuracy at low precision. As shown in Table 2, under uniform INT2 quantization with group size 16 and without rotation, Previous-loop reconstruction with unit scaling achieves 63.6% MATH500 accuracy. Switching to the Last-loop reference improves accuracy to 67.6%, and adding LS scaling further increases it to 69.0%. Together, these design choices provide a more accurate representation of the inter-loop residuals, allowing RESIDUALQUANT to operate effectively at INT2. On MATH500, RESIDUALQUANT incurs less than 10% accuracy degradation from BF16 under, whereas FlashLoop reports a sharp accuracy drop to below 20% when its residual quantization is reduced to INT2.

Table 10: Combining RESIDUALQUANT with sparse KV updates and attention. MATH500 accuracy on Ouro-1.4B and average KV bitwidth during prefill and decode. All methods use the same sparse KV update and attention mechanisms from FlashLoop. For RESIDUALQUANT, g denotes the INT2 residual group size, while INT4 anchors use $g = 32$. Average bitwidth includes quantized values, scales, offsets, and LS coefficients, but excludes recent BF16 states.

Method	BF16 KV tokens	Average Bitwidth		Accuracy (%)
		Prefill	Decode	
BF16 + sparse	All	9.40	16.00	76.0
FlashLoop	Recent 64–127	2.64	4.50	72.8
RESIDUALQUANT ($g = 16$) + sparse	Recent 64	2.18	3.47	74.4
RESIDUALQUANT ($g = 32$) + sparse	Recent 64	2.01	3.09	73.4

Orthogonal integration with sparse updates and attention. RESIDUALQUANT is orthogonal to FlashLoop’s sparse KV update and attention mechanisms. Here, we further integrate RESIDUALQUANT with FlashLoop’s sparse update and attention mechanisms to achieve higher accuracy with a smaller KV cache footprint.

We evaluate Ouro-1.4B on all 500 MATH500 problems using an NVIDIA A100-SXM4-40GB, matching the GPU model reported by FlashLoop. All methods use the same model revision, zero-shot plain prompts, greedy decoding, seed 0, and a 2,048-token generation limit, with answers scored using Math-Verify. For FlashLoop, we directly use its public codebase¹ and default INT4 quantization with group size $g = 64$. For RESIDUALQUANT, we use INT2/4 with INT2 group sizes $g \in \{16, 32\}$ and INT4 group size $g = 32$. FlashLoop’s block-wise flushing retains between 64 and 127 recent tokens in BF16; for RESIDUALQUANT, we retain exactly 64 recent tokens in BF16 for a fair comparison. We directly apply the sparse update and attention mechanisms from the FlashLoop codebase to RESIDUALQUANT. Under sparsification, the final loop may not be retained for every token; in such cases, we use the last non-pruned loop as the INT4 anchor and represent the remaining retained loops with INT2 residuals.

As shown in Table 10, combining sparse updates and attention with RESIDUALQUANT improves both accuracy and KV storage over FlashLoop. With $g = 16$, RESIDUALQUANT reaches 74.4% accuracy compared with 72.8% for FlashLoop, while reducing average bitwidth from 2.64 to 2.18 for prefill and from 4.50 to 3.47 for decode. With $g = 32$, it further reduces bitwidth to 2.01 and 3.09, respectively, while still improving accuracy to 73.4%. These results show that RESIDUALQUANT provides a stronger quantization method even when combined with FlashLoop’s sparse update and attention mechanisms, further improving the accuracy–memory tradeoff.

¹<https://github.com/Superone77/FlashLoop>